

一、讲述 Qt 信号槽机制与优势与不足

优点： ①类型安全。需要关联的信号槽的签名必须是等同的。即信号的类型和参数个数同接受该信号的槽的参数类型和参数个数相同。若信号和槽签名不一致，编译器会报错。

②松散耦合。信号和槽机制减弱了 Qt 对象的耦合度。激发信号的 Qt 对象无需知道是哪个对象的那个信号槽接收它发出的信号，它只需在适当的时间发送适当的信号即可，而不需要关心是否被接受和那个对象接受了。Qt 就保证了适当的槽得到了调用，即使关联的对象在运行时被删除。程序也不会奔溃。

③灵活性。一个信号可以关联多个槽，或多个信号关联同一个槽。

不足： 速度较慢。与回调函数相比，信号和槽机制运行速度比直接调用非虚函数慢 10 倍。

原因：①需要定位接收信号的对象。②安全地遍历所有关联槽。③编组、解组传递参数。④多线程的时候，信号需要排队等待。（然而，与创建对象的新操作及删除对象的 delete 操作相比，信号和槽的运行代价只是他们很少的一部分。信号和槽机制导致的这点性能损耗，对实时应用程序是可以忽略的。）

二、Qt 信号和槽的本质是什么

回调函数。信号或是传递值，或是传递动作变化；槽函数响应信号或是接收值，或者根据动作变化来做出对应操作。

三、描述 QT 中的文件流(QTextStream)和数据流(QDataStream)的区别

文件流(QTextStream)。操作轻量级数据（int, double, QString）数据写入文本文件中以后以文本的方式呈现。

数据流(QDataStream)。通过数据流可以操作各种数据类型，包括对象，存储到文件中数据为二进制。

文件流，数据流都可以操作磁盘文件，也可以操作内存数据。通过流对象可以将对象打包到内存，进行数据的传输。

四、描述 QT 的 TCP 通讯流程

服务端：（QTcpServer）

- ①创建 QTcpServer 对象
- ②监听 list 需要的参数是地址和端口号
- ③当有新的客户端连接成功回发送 newConnect 信号
- ④在 newConnection 信号槽函数中，调用 nextPendingConnection 函数获取新连接 QTcpSocket 对象
- ⑤连接 QTcpSocket 对象的 readRead 信号
- ⑥在 readRead 信号的槽函数使用 read 接收数据
- ⑦调用 write 成员函数发送数据

```

1  Widget::Widget(QWidget *parent) :
2      QWidget(parent),
3      ui(new Ui::Widget)
4  {
5      ui->setupUi(this);
6      tcpServer = new QTcpServer;
7      tcpServer->listen(QHostAddress("192.168.0.111"),1234);
8      connect(tcpServer,SIGNAL(newConnection()),this,SLOT(new_connect()));
9  }
10
11 Widget::~Widget()
12 {
13     delete ui;
14 }
15
16 void Widget::new_connect()
17 {
18     qDebug("--new connect--");
19     QTcpSocket* tcpSocket = tcpServer->nextPendingConnection();
20     connect(tcpSocket,SIGNAL(readyRead()),this,SLOT(read_data()));
21     socketArr.push_back(tcpSocket);
22 }
23
24
25 void Widget::read_data()
26 {
27     for(int i=0; i<socketArr.size(); i++)
28     {
29         if(socketArr[i]->bytesAvailable())
30         {
31             char buf[256] = {};
32             socketArr[i]->read(buf,sizeof(buf));
33             qDebug("---read:%s---",buf);
34         }
35     }
36 }

```

客户端：（QTcpSocket）

- ①创建 QTcpSocket 对象
- ②当对象与 Server 连接成功时会发送 connected 信号
- ③调用成员函数 connectToHost 连接服务器，需要的参数是地址和端口号
- ④connected 信号的槽函数开启发送数据
- ⑤使用 write 发送数据，read 接收数据

```

1  Widget::Widget(QWidget *parent) :
2      QWidget(parent),
3      ui(new Ui::Widget)
4  {
5      ui->setupUi(this);
6      tcpSocket = new QTcpSocket;
7      connect(tcpSocket,SIGNAL.connected()),this,SLOT(connect_success());
8      tcpSocket->connectToHost("172.20.10.3",1234);
9  }
10
11 Widget::~Widget()
12 {
13     delete ui;
14 }
15
16 void Widget::on_send_clicked()
17 {
18     std::string msg = ui->msg->text().toStdString();
19     int ret = tcpSocket->write(msg.c_str(),msg.size()+1);
20     qDebug("---send:%d---",ret);
21 }
22
23 void Widget::connect_success()
24 {
25     ui->send->setEnabled(true);
26 }

```

五、 描述 UDP 之 UdpSocket 通讯

UDP (User Datagram Protocol 即用户数据报协议) 是一个轻量级的, 不可靠的, 面向数据报的无连接协议。在网络质量令人十分不满意的环境下, UDP 协议数据包丢失严重。由于 UDP 的特性: 它不属于连接型协议, 因而具有资源消耗小, 处理速度快的优点, 所以通常音频、视频和普通数据在传送时使用 UDP 较多, 因为它们即使偶尔丢失一两个数据包, 也不会对接收结果产生太大影响。所以 QQ 这种对保密要求并不太高的聊天程序就是使用的 UDP 协议。

在 Qt 中提供了 QUdpSocket 类来进行 UDP 数据报 (datagrams) 的发送和接收。Socket 简单地说, 就是一个 IP 地址加一个 port 端口。

流程: ①创建 QUdpSocket 套接字对象 ②如果需要接收数据, 必须绑定端口 ③发送数据用 writeDatagram, 接收数据用 readDatagram。

六、多线程使用使用方法

方法一：①创建一个类从 QThread 类派生②在子线程类中重写 run 函数，将处理操作写入该函数中 ③在主线程中创建子线程对象，启动子线程，调用 start() 函数

方法二：①将业务处理抽象成一个业务类，在该类中创建一个业务处理函数②在主线程中创建一 QThread 类对象 ③在主线程中创建一个业务类对象 ④将业务类对象移动到子线程中 ⑤在主线程中启动子线程 ⑥通过信号槽的方式，执行业务类中的业务处理函数

多线程使用注意事项：

- * 1. 业务对象，构造的时候不能指定父对象
- * 2. 子线程中不能处理 ui 窗口(ui 相关的类)
- * 3. 子线程中只能处理一些数据相关的操作，不能涉及窗口

七、多线程下，信号槽分别在什么线程中执行，如何控制

可以通过 `connect` 的第五个参数进行控制信号槽执行时所在的线程

`connect` 有几种连接方式，直接连接和队列连接、自动连接

直接连接 (`Qt::DirectConnection`)：信号槽在信号发出者所在的线程中执行

队列连接 (Qt::QueuedConnection): 信号在信号发出者所在的线程中执行, 槽函数在信号接收者所在的线程中执行

自动连接 (Qt::AutoConnection): 多线程时为队列连接函数, 单线程时为直接连接函数。

八、自定义控件流程

继承需要自定义的控件类, 如 QPushButton; 从外观设计上: QSS、继承绘制函数重绘、继承 QStyle 相关类重绘、组合拼装等等; 从功能行为上: 重写事件函数、添加或者修改信号和槽等等。

九、对 QObject 的理解

QObject 类是 Qt 所有类的基类。

QObject 是 Qt 对象模型的核心。这个模型的中心要素就是一种强大的叫做信号与槽无缝对象沟通机制。你可以用 connect() 函数来把一个信号连接到槽, 也可以用 disconnect() 函数来破坏这个连接。为了避免永无止境的通知循环, 你可以用 blockSignal() 函数来暂时阻塞信号。保护函数 connectNotify() 和 disconnectNotify() 可以用来跟踪连接。

对象树都是通过 QObject 组织起来的, 当以一个对象作为父类创建一个新的对象时, 这个新对象会被自动加入到父类的 children() 队列中。这个父类有子类的所有权。能够在父类的析构函数中自动删除子类。可以通过 findChild() 和 findChildren() 函数来寻找子类。

每个对象都有一个对象名称 objectName(), 而且它的类名也可以通过 metaObject() 函数。你可以通过 inherits() 函数来决定一个类是否继承其他的类。当一个对象被删除时, 它会发射 destroy() 信号, 你可以抓住这个信号避免某些事情。

对象可以通过 event() 函数来接收事情以及过滤来自其他对象的事件。就好比 installEventFilter() 函数和 eventFilter() 函数。childEvent() 函数能够重载实现子对象的事件。

QObject 还提供了基本的时间支持, QTimer 类 提高了更高层次的时间支持。

任何对象要实现信号与槽机制，Q_OBJECT 宏都是强制的。你也需要在源原件上运行元对象编译器。不管是否真正用到信号与槽机制，最好在所有 QObject 子类使用 Q_OBJECT 宏，以避免出现一些不必要的错误。

所有的 Qt widgets 都是基础 QObject。如果一个对象是 widget, 那么 isWidgetType() 函数就能判断出。

十、Qt 自定义一个信号槽，触发这个信号，Qt 多个信号如何关联一并处理；

第一种方法：

在发送信号时，也发送一个 int 类型数字，或者说标志，这样在槽函数触发是可以知道是哪个信号发出的；

第二种方法：

在槽函数内有获取发送信号的函数，通过 sender() 函数获取发送信号；

十一、Qt 如果一个信号的处理方法一直未被执行有哪些可能性；

断开了，连接的时候失败了，多线程的时候在排队或者启动锁死了。

十二、在 Qt5 的信号处理中如何使用 lambda 机制（可以代码示例）；

信号定义了，但是不写对应槽函数，直接将函数写到槽的位置。

```
connect(musicPlayer, SIGNAL(positionChanged(qint64)), this, SLOT(slotReflushStartTime(qint64)));
```

```
connect(musicPlayer, SIGNAL(positionChanged(qint64)), slotReflushStartTime(qint64));
```

直接就是将对象都不写了，直接写个函数。

十三、段错误一般是什么原因造成的，如何快速排查；

一般是指针的问题，出现野指针空指针；点灯或者用 Debug 去排查问题。

十四：如图

9. 代码推演 1

```
#include <QtGui/QApplication>
#include <QtGui/QLabel>
#include <QtGui/QWidget>
int main(int argc, char* argv[]){
    QApplication hwApp(argc, argv);
    QLabel hwLabel("Hello world");
    QWidget window;
    hwLabel.setParent(&window);
    window.show();
    return hwApp.exec();
}
```

运行后有什么问题吗？

代码推演 2

```
class A{
public:
    void print () {cout<<"Hello, this is A"<<endl;}
};
class B{
public:
    void print () {cout<<"Hello, this is B"<<endl;}
};
class C : public A, public B{
public:
    void disp () {
        .....
    }
}
```

类 C 中 disp 函数, 如果想调用类 A 中的函数 print, 应该如果编写？

第一个：在关闭界面时会异常退出，由于是在栈区创建之后会自己释放，所以程序结束后创建先释放，所以这里是 widget 后创建先释放，导致 label 还没释放但是承载他的 widget 已经连带着自己和 label 释放了，但是 label 还做了内存管理，内存管理不会去判断是否已经释放了，就会出现自己再释放一次的问题；所以解决方法就是将 widget 先创建。

第二个：A: : printf () ;

十五、Qt 三大核心机制

21.1 信号槽

信号槽的五种连接方式

```
1      enum ConnectionType {
2          AutoConnection,      // 默认连接方式 单线程 DirectConnection 多线程 DirectConn
3          DirectConnection,    // 单线程
4          QueuedConnection,    // 多线程异步
5          BlockingQueuedConnection, // 多线程同步
6          UniqueConnection = 0x80 // 唯一连接，用来防止重复连接
7      };
8
```

21.2 元对象系统

元对象系统分为三大类:QObject 类、Q_OBJECT 宏和元对象编译器 moc

Qt 的类包含 Q_OBJECT 宏 moc 编译器会对该类编译成标准的 C++ 代码

21.3 事件模型

事件的创建

鼠标事件，键盘事件，窗口调整事件，模拟事件

事件的交付

Qt 通过调用虚函数 QObject::event() 来交付事件。

事件循环模型

主事件循环通过调用 QApplication::exec() 启动，

随着 QApplication::exit() 结束，

本地的事件循环可用利用 QEventLoop 构建。

一般来说，事件是由触发当前的窗口系统产生的，但也可以通过使用 QApplication::sendEvent() 和

QCoreApplication::postEvent() 来手工产生事件。需要说明的是
QCoreApplication::sendEvent() 会立即发送事件，
QCoreApplication::postEvent() 则会将事件放在事件队列中分发。

自定义事件

十六、Qt 对象树

QT 提供了对象树机制，能够自动、有效的组织和管理继承自 `QObject` 的对象。

每个继承自 `QObject` 类的对象通过它的对象链表(`QObjectList`)来管理子类对象，当用户创建一个子对象时，其对象链表相应更新子类对象的信息，对象链表可通过 `children()` 获取。

当父类对象析构的时候，其对象链表中的所有(子类)对象也会被析构，父对象会自动，将其从父对象列表中删除，QT 保证没有对象会被 `delete` 两次。开发中手动回收资源时建议使用 `deleteLater` 代替 `delete`, 因为 `deleteLater` 多次是安全的。

十七、描述 `QTextStream`(文件流)和 `QDataStream`(数据流)的区别

文本流用来操作轻量级的数据，比如内置的 `int`、`QString` 等，写入文件后以文本的方式呈现

数据流，可以操作各种类型数据

总之，两者都可以进行操作磁盘文件以及内存数据。

十八、信号槽的四种写法和五种连接方式？

`connect`(信号发出者, 信号, 信号接收者, 槽, 连接方式(隐藏默认自动连接))//
五个参数

四种写法:

1. 用宏:

```
connect(this, SIGNAL(clicked()), this, SLOT(colse())); //连接方式(隐藏默认自动连接))
```

2. 用函数指针:

```
connect(this, &mainwindow::my_signal, this, &mainwindow::my_slot);
```

3. 用重载函数指针 `QOverload`

```
connect(this, QOverload<参数>::of(&mainwindow::my_signal), this, QOverload<参数>::of(&mainwindow::my_slot));
```

4. lambda 表达式(匿名函数) 匿名函数代替槽

```
connect(this, &mainwindow::my_signal, this, [=] {qDebug() << 100;});
```

连接方式: 自动连接(默认连接方式)

直接连接(用于单线程, 自动匹配)

队列(用于多线程也可用于单线程, 自动匹配)

阻塞队列(跨线程, 多线程)

唯一连接(跨线程, 多线程)

十九、qt 模型

答: Qt 中的 View 主要有三种 QListView, QTreeView, QTableView

而对应的 Model 是: QStringListModel, QAbstractItemModel ,

QStandardItemModel。

抽象 标准

二十、Qt 中的 MVD 了解吧?

Qt 的 MVD 包含三个部分 Model (模型), View (视图), 代理 (Delegate)。Model 否则保存数据, View 负责展示数据, Delegate 负责 Item 样式绘制或处理输入。这三部分通过信号槽来进行通信, 当 Model 中数据发生变化时将会发送信号到 View, 在 View 中编辑数据时, Delegate 负责将编辑状态发送给 Model 层。基类分别为 QAbstractItemModel、QAbstractItemView、QAbstractItemDelegate。Qt 中提供了默认实现的 MVD 类, 如 QTableWidget、QListWidget、QTreeWidget 等。

二十一、QT 如果要进行网络编程首先需要在.pro 中添加如下代码 QT network

在头文件中包含相关头文件

```
include QHostInfo
```

```
include QNetworkInterface
```

因无法显示 略去#与)

2、QT 的 UdpSocket 接收消息使用原则

第一步 new 一个 UdpSocket

第二步 调用 UdpSocket 的 bind 方法 同时指定端口号

第三步 使用 connect 将接收消息函数和 UdpSocket 对象做关联

第四步 在接受消息槽函数当中调用 readDatagram 接收消息

更多面试题目：

- 1、qt 如何实现类似 QQ 登录窗口的翻转
- 2、窗口圆角如何实现
- 3、如何实现 QQ 换肤功能
- 4、按比例拉大窗口 部分控件的大小和控件间距不变
- 5、QT 的信号与槽，有哪几种连接方式，对应的应用场景是什么?(第五个参数)
- 6、QWidget 和 QML 的技术本质和使用上，有什么区别?

7、软件如果出现问题，如何去定位的，如何处理的?静态扫描和动态检测，有哪些方法.

8、用 QT 实现一个三角形的按钮，会如何实现?

9、Qt Remote Object 的序列化与反序列化

10、QString 和 string 的区别， 的实现方式

11、QWidget 和 QML 的区别，在渲染层面

12、windows 系统下，是怎么实现窗口刷新(窗口刷新机制);是立即刷新，还是异步刷新;每次我需要一个窗口刷新，他都能立马刷新吗

13、父子窗口间的刷新管理? 兄弟窗口间的刷新管理?如何让子窗口刷新，父窗口不刷新?

14、兄弟窗口，想刷新他们的重叠部分，请问流程是什么样的，刷新的顺序是什么样的?

15、QT 的指针指针，QSharePoint 和 shared_ptr 有什么区别，weak_ptr 呢?

16、QShareDataPoint 作用

17、创建的对象有几种方式，有什么区别

18、请简述如何实现 QQ 两个客户端的私聊功能?

19、使用过什么数据库，qt 如何操作数据库（增删改查等）?

20、如何看待 Qt 的应用场景?

21、什么是 UI 线程，UI 线程阻塞后会怎样?

22、描述 QT 中的文件流(QTextStream)和数据流(QDataStream)的区别, 他们都能帮助我们完成一些什么事情.

23、描述过程, 如何实现一个自定义按钮, 使其在光标进入,按下,离开三种状态下显示不同的图片.

24、Qt Socket 通信的过程。

25、遇到过中文乱码吗？

26、程序打包说一下

27、谈一下对 Qt 元对象系统的了解

28、Qt 创建界面的几种方法？

29、什么是事件循环？(`QCoreApplication::exec()` `QCoreApplication::exit()`)

30、什么是面向对象，面向对象的三大特征是什么？

31、你能用几种方法修改 `QPushButton` 的大小，文字颜色等属性。

32、常用的布局有几种，如何自适应缩放？

33、如何将键盘和鼠标的相关操作过滤出来并关联到自己想要执行的函数上？

34、为什么要异步刷新，如何异步刷新？

35、多线程如何通信？

